

**Universidade de Brasília**

Departamento de Ciência da Computação

Organização e Arquitetura de Computadores – Grupo 01, 2026/1

Prof. Marcus Vinicius Lamar

## **Laboratório 02**

Guilherme Nascimento - 222001449

Letícia Brito - 242039381

Halycia de Oliveira Fonseca - 221037625

Eduardo Pereira de Sousa - 231018937

Matheus Chagas Lopes - 222011599

Pedro Fernandes de Oliveira - 231006177

## 2) Unidade Lógico-Aritmética de Inteiros

### Questão 2 - item a

A partir da análise da descrição em Verilog da ULA de inteiros fornecida, foi possível descrever suas funções e extrair a tabela de códigos para cada operação executada pelo módulo. O circuito é orientado por um sinal de controle de 5 bits (iControl), responsável por determinar qual operação lógica ou aritmética será aplicada aos operandos iA e iB.

Para complementar a análise, o arquivo ALU.v foi definido como toplevel do projeto e devidamente compilado. O circuito sintetizado correspondente à estrutura da tabela foi visualizado e documentado por meio da ferramenta Tools / Netlist Viewers / RTL viewer, conforme a figura a seguir:

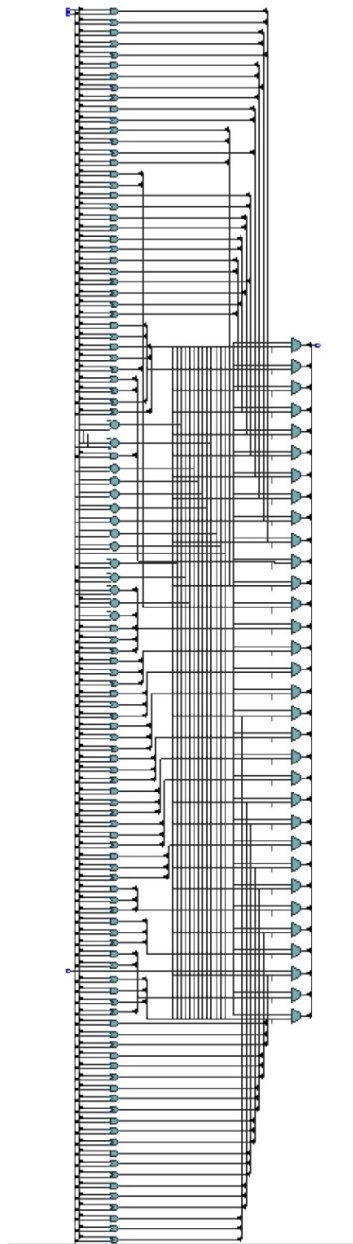


Figure 1: Circuito lógico da ULA gerado no RTL Viewer.

Table 1: Operações da Unidade Lógico Aritmética (ULA)

| Código (Binário) | Decimal | Operação Executada                           | Descrição da Função                                         |
|------------------|---------|----------------------------------------------|-------------------------------------------------------------|
| 00000            | 5'd0    | $iA \& iB$                                   | AND lógico bit a bit                                        |
| 00001            | 5'd1    | $iA \mid iB$                                 | OR lógico bit a bit                                         |
| 00010            | 5'd2    | $iA \wedge iB$                               | XOR lógico bit a bit                                        |
| 00011            | 5'd3    | $iA + iB$                                    | Adição                                                      |
| 00100            | 5'd4    | $iA - iB$                                    | Subtração                                                   |
| 00101            | 5'd5    | $iA < iB$                                    | Set Less Than (Compara c/sinal)                             |
| 00110            | 5'd6    | $\text{unsigned}(iA) < \text{unsigned}(iB)$  | Set Less Than Sem sinal                                     |
| 00111            | 5'd7    | $iA \ll iB[4:0]$                             | Shift Left Logical                                          |
| 01000            | 5'd8    | $iA \gg iB[4:0]$                             | Shift Right Logical                                         |
| 01001            | 5'd9    | $iA \ggg iB[4:0]$                            | Shift Right Arithmetic                                      |
| 01010            | 5'd10   | $iB$                                         | Passa o valor de B para a saída                             |
| 01011            | 5'd11   | $\text{mul}[31:0]$                           | Multiplicação (32 bits menos sig.)                          |
| 01100            | 5'd12   | $\text{mul}[63:32]$                          | Multiplicação (32 bits mais sig., c/ sinal)                 |
| 01101            | 5'd13   | $\text{mulu}[63:32]$                         | Multiplicação (32 bits mais sig., s/ sinal)                 |
| 01110            | 5'd14   | $\text{mulsu}[63:32]$                        | Multiplicação ( $A \text{ c/ sinal} * B \text{ s/ sinal}$ ) |
| 01111            | 5'd15   | $iA / iB$                                    | Divisão com sinal                                           |
| 10000            | 5'd16   | $\text{unsigned}(iA) / \text{unsigned}(iB)$  | Divisão sem sinal                                           |
| 10001            | 5'd17   | $iA \% iB$                                   | Resto da divisão (Módulo) com sinal                         |
| 10010            | 5'd18   | $\text{unsigned}(iA) \% \text{unsigned}(iB)$ | Resto da divisão (Módulo) sem sinal                         |
| 11111            | 5'd31   | ZERO                                         | Saída zerada (Inválido)                                     |

## Questão 2 - item b

### Cenário 1: Valores Representativos ( $A = 1$ , $B = 2$ )

Neste primeiro cenário, os operandos foram definidos com valores inteiros pequenos e positivos para verificar o comportamento padrão da ULA nas operações aritméticas e lógicas básicas. O objetivo é validar o caminho de dados principal da arquitetura em condições normais de operação, caracterizando um caso de teste com valores comuns. Através destes valores, é possível confirmar facilmente e de forma analítica a exatidão das funções lógicas bit a bit (AND, OR, XOR), as operações de deslocamento (Shifts) e a correta execução da adição e subtração elementares.

### Cenário 2: Caso Singular - Divisão por Zero ( $A = 1$ , $B = 0$ )

Este teste foi estruturado para avaliar o comportamento do hardware frente a uma exceção matemática, caracterizando um dos resultados singulares exigidos pela especificação da prática. Ao forçar o divisor (operando B) a zero nas operações de divisão com e sem sinal (OPDIV e OPDIVU), observa-se que a ULA não interrompe o processamento. Em vez disso, a saída resulta em todos os 32 bits em nível lógico alto (1). Em notação de complemento de dois, isto equivale ao valor decimal -1, o que obedece à convenção padrão de tratamento de divisão por zero em arquiteturas como a RISC-V. As operações

Table 2: Resultados da ULA para os valores representativos (A=1, B=2)

| Nome da Operação | Resultado (A=1, B=2) em Binário (32 bits) |
|------------------|-------------------------------------------|
| OPAND            | 00000000000000000000000000000000          |
| OPOR             | 00000000000000000000000000000011          |
| OPXOR            | 00000000000000000000000000000011          |
| OPADD            | 00000000000000000000000000000011          |
| OPSUB            | 11111111111111111111111111111111          |
| OPSLT            | 00000000000000000000000000000001          |
| OPSLTU           | 00000000000000000000000000000001          |
| OPSLL            | 00000000000000000000000000000100          |
| OPSRL            | 00000000000000000000000000000000          |
| OPSRA            | 00000000000000000000000000000000          |
| OPLUI            | 00000000000000000000000000000010          |
| OPMUL            | 00000000000000000000000000000010          |
| OPMULH           | 00000000000000000000000000000000          |
| OPMULHU          | 00000000000000000000000000000000          |
| OPMULHSU         | 00000000000000000000000000000000          |
| OPDIV            | 00000000000000000000000000000000          |
| OPDIVU           | 00000000000000000000000000000000          |
| OPREM            | 00000000000000000000000000000001          |
| OPREMU           | 00000000000000000000000000000001          |
| OPNULL           | 00000000000000000000000000000000          |

de resto da divisão (OPREM e OPREMU) retornam o próprio valor do dividendo.

Table 3: Resultados da ULA para o caso singular: Divisão por Zero (A=1, B=0)

| Nome da Operação | Resultado (A=1, B=0) em Binário (32 bits) |
|------------------|-------------------------------------------|
| OPAND            | 00000000000000000000000000000000          |
| OPOR             | 00000000000000000000000000000001          |
| OPXOR            | 00000000000000000000000000000001          |
| OPADD            | 00000000000000000000000000000001          |
| OPSUB            | 00000000000000000000000000000001          |
| OPSLT            | 00000000000000000000000000000000          |
| OPSLTU           | 00000000000000000000000000000000          |
| OPSLL            | 00000000000000000000000000000001          |
| OPSRL            | 00000000000000000000000000000001          |
| OPSRA            | 00000000000000000000000000000001          |
| OPLUI            | 00000000000000000000000000000000          |
| OPMUL            | 00000000000000000000000000000000          |
| OPMULH           | 00000000000000000000000000000000          |
| OPMULHU          | 00000000000000000000000000000000          |
| OPMULHSU         | 00000000000000000000000000000000          |
| OPDIV            | 11111111111111111111111111111111          |
| OPDIVU           | 11111111111111111111111111111111          |
| OPREM            | 00000000000000000000000000000001          |
| OPREMU           | 00000000000000000000000000000001          |
| OPNULL           | 00000000000000000000000000000000          |

### Cenário 3: Limite de Bits, Comportamento de Sinal e Overflow (A = -1, B = 1)

O terceiro cenário explora as fronteiras da representação binária de 32 bits e a forma como a ULA diferencia as instruções executadas com sinal (signed) e sem sinal (unsigned). O operando A foi preenchido com todos os bits em nível alto (32'hFFFFFFF). Em complemento de dois, isso representa o valor -1; no entanto, em lógica sem sinal, representa o valor máximo possível suportado pela arquitetura. Este teste singular é fundamental para comprovar o funcionamento de três situações críticas:

1. Comparação: O contraste entre a instrução OPSLT (onde -1 > 1 resulta em verdadeiro, saída 1) e a instrução OPSLTU (onde o valor máximo sem sinal não é menor que 1, resultando em falso, saída 0).

2. Overflow Aritmético: Na operação de adição (OPADD), a soma destes valores resulta em zero puro, evidenciando o descarte do carry out (estouro da capacidade do registrador de 32 bits).

3. Deslocamento Aritmético vs. Lógico: Diferencia o preenchimento de bits de sinal no Shift Right Arithmetic (OPSRA) em relação ao preenchimento com zeros no Shift Right Logical (OPSRL).

Table 4: Resultados da ULA para teste de limite de bits/sinal (A=-1, B=1)

| Nome da Operação | Resultado (A=-1, B=1) em Binário |
|------------------|----------------------------------|
| OPAND            | 00000000000000000000000000000001 |
| OPOR             | 11111111111111111111111111111111 |
| OPXOR            | 11111111111111111111111111111110 |
| OPADD            | 00000000000000000000000000000000 |
| OPSUB            | 11111111111111111111111111111110 |
| OPSLT            | 00000000000000000000000000000001 |
| OPSLTU           | 00000000000000000000000000000000 |
| OPSLL            | 11111111111111111111111111111110 |
| OPSRL            | 01111111111111111111111111111111 |
| OPSRA            | 11111111111111111111111111111111 |
| OPLUI            | 00000000000000000000000000000001 |
| OPMUL            | 11111111111111111111111111111111 |
| OPMULH           | 11111111111111111111111111111111 |
| OPMULHU          | 00000000000000000000000000000000 |
| OPMULHSU         | 11111111111111111111111111111111 |
| OPDIV            | 11111111111111111111111111111111 |
| OPDIVU           | 11111111111111111111111111111111 |
| OPREM            | 00000000000000000000000000000000 |
| OPREMU           | 00000000000000000000000000000000 |
| OPNULL           | 00000000000000000000000000000000 |

## 1 Questão - 2 item c)

| Operação                 | ALMs        | Registradores | Memória (bits) | Blocos DSP |
|--------------------------|-------------|---------------|----------------|------------|
| OPAND                    | 65          | 0             | 0              | 0          |
| OPOR                     | 65          | 0             | 0              | 0          |
| OPXOR                    | 65          | 0             | 0              | 0          |
| OPADD                    | 65          | 0             | 0              | 0          |
| OPSUB                    | 65          | 0             | 0              | 0          |
| OPSLT                    | 79          | 0             | 0              | 0          |
| OPSLTU                   | 79          | 0             | 0              | 0          |
| OPSLL                    | 120         | 0             | 0              | 0          |
| OP SRL                   | 121         | 0             | 0              | 0          |
| OPSRA                    | 118         | 0             | 0              | 0          |
| OPLUI                    | 49          | 0             | 0              | 0          |
| OPMUL                    | 56          | 0             | 0              | 2          |
| OPMULH                   | 72          | 0             | 0              | 3          |
| OPMULHU                  | 72          | 0             | 0              | 3          |
| OPMULHSU                 | 88          | 0             | 0              | 6          |
| OPDIV                    | 704         | 0             | 0              | 0          |
| OPDIVU                   | 633         | 0             | 0              | 0          |
| OPREM                    | 728         | 0             | 0              | 0          |
| OPREMU                   | 652         | 0             | 0              | 0          |
| OPNULL                   | 49          | 0             | 0              | 0          |
| <b>Soma das Isoladas</b> | <b>3945</b> | <b>0</b>      | <b>0</b>       | <b>14</b>  |
| <b>ULA Total</b>         | <b>3033</b> | <b>0</b>      | <b>0</b>       | <b>12</b>  |

Table 5: Requisitos físicos da implementação da ULA por operação e do circuito total

A soma dos ALMs de todas as operações isoladas totalizou 3.945 ALMs, enquanto a síntese da ULA completa consumiu apenas 3.033 ALMs. Essa diferença de 912 ALMs é resultado da eficiência do Quartus em aplicar a otimização de Compartilhamento de Recursos. Ao invés de replicar blocos para cada instrução, o compilador identifica redundâncias em operações semelhantes como a adição e subtração, e as reaproveita para diminuir o uso de recursos.

A diferença de dois blocos DSP da ULA completa e da soma das operações individuais se deve ao fato de que as operações OPMUL, OPMULH, OPMULHU e OPMULHSU compartilham as mesmas entradas de 32 bits, e a matriz inicial de produtos parciais (bits inferiores) é igual para todas. Essa igualdade permite que o compilador ramifique o circuito apenas nos estágios finais para processar os bits de sinal superiores, economizando blocos DSP.

O tamanho total da ULA é definido majoritariamente pelas operações de divisão, resto e multiplicação, que impactam recursos diferentes:

- Consumo de Lógica (ALMs): As operações de Divisão e Resto (OPDIV, OPDIVU, OPREM, OPREMU) possuem o maior impacto no consumo de lógica genérica, exigindo um total de 2.717 ALMs, aproximadamente 68,9% do consumo individual total.
- Consumo de IP Dedicado (DSP): As operações de Multiplicação (OPMUL, OPMULH, OPMULHU e OPMULHSU) são responsáveis por 100% do consumo dos blocos DSP da ULA.

## 2 Questão 2 - item d

### I) e II) Identificação das Operações com Maior Tempo de Atraso

As funções que apresentaram os maiores atrasos de propagação ( $t_{pd}$ ) foram as operações de resto (*OPREM*), com 94,185 ns pelo caminho  $iB[7] \rightarrow oResult[24]$ , e divisão (*OPDIV*), com 92,628 ns pelo caminho  $iB[0] \rightarrow oResult[31]$ . Ambas exibem um tempo de estabilização de sinal drasticamente superior a todas as demais operações lógicas e aritméticas da ULA, consolidando-se como os principais gargalos temporais do circuito.

### III) Impacto na ULA Total

Em circuitos puramente combinatórios, o atraso global do bloco é delimitado estritamente pela rota mais lenta. Por isso, o impacto das funções de divisão e resto reflete diretamente no atraso da ULA Total (105,323 ns), que compartilha o mesmo pino de origem ( $iB[7]$ ) em sua rota de maior atraso. O acréscimo de aproximadamente 11 ns observado no circuito completo em relação à operação isolada deve-se ao *overhead* de propagação introduzido pelas camadas de portas lógicas do multiplexador de saída.

Table 6: Requerimentos Temporais para a Ula Total e suas Operações

| Operação         | Maior Tempo de Atraso (ns) | Caminho de Maior Atraso                           |
|------------------|----------------------------|---------------------------------------------------|
| OPAND            | 2,312                      | $iA[31] \rightarrow oResult[31]$                  |
| OPOR             | 2,312                      | $iA[31] \rightarrow oResult[31]$                  |
| OPXOR            | 2,312                      | $iA[31] \rightarrow oResult[31]$                  |
| OPADD            | 2,865                      | $iB[2] \rightarrow oResult[26]$                   |
| OPSUB            | 2,939                      | $iB[3] \rightarrow oResult[24]$                   |
| OPSLT            | 4,490                      | $iA[18] \rightarrow oResult[0]$                   |
| OPSLTU           | 4,490                      | $iA[18] \rightarrow oResult[0]$                   |
| OPSLI            | 4,166                      | $iA[21] \rightarrow oResult[25]$                  |
| OPSRL            | 4,237                      | $iB[1] \rightarrow oResult[2]$                    |
| OPSRA            | 4,099                      | $iA[31] \rightarrow oResult[5]$                   |
| OPLUI            | 0,516                      | $iB[29] \rightarrow oResult[29]$                  |
| OPMUL            | 7,547                      | $iB[9] \rightarrow oResult[28]$                   |
| OPMULH           | 8,165                      | $iA[18] \rightarrow oResult[22]$                  |
| OPMULHU          | 8,535                      | $iA[16] \rightarrow oResult[25]$                  |
| OPMULHSU         | 11,571                     | $iB[20] \rightarrow oResult[27]$                  |
| OPDIV            | 92,628                     | $iB[0] \rightarrow oResult[31]$                   |
| OPDIVU           | 81,512                     | $iB[26] \rightarrow oResult[0]$                   |
| OPREM            | 94,185                     | $iB[7] \rightarrow oResult[24]$                   |
| OPREMU           | 87,490                     | $iB[27] \rightarrow oResult[12]$                  |
| <b>ULA Total</b> | <b>105,323</b>             | <b><math>iB[7] \rightarrow oResult[19]</math></b> |

## Questão 2 - item e

Os resultados obtidos confirmam o funcionamento da ULA em diferentes condições lógicas e aritméticas conforme a arquitetura RISC-V implementada e os cenários propostos na resposta 2b: no primeiro cenário, a operação de soma entre  $A = 1$  e  $B = 2$  resultou corretamente em 3 (000003<sub>16</sub>); no segundo, a soma de  $A = 1$  e  $B = 0$  manteve o valor 1 (000001<sub>16</sub>); e no terceiro, a operação com  $A = -1$

( $FFFFFFFF_{16}$  em 32 bits) e  $B = 1$  resultou em 0 ( $000000_{16}$ ), confirmando que a extensão de sinal realizada pelo TopDE.v atua corretamente em complemento de dois, permitindo que a ULA processe números negativos e realize cálculos de forma consistente. É possível observar todos os resultados esperados neste vídeo.

link do vídeo: <https://youtu.be/wgQnKX9Uqcw>

## Questao 2 - item f

Na descrição Verilog original fornecida para o laboratório, a ULA instanciou estruturalmente três multiplicadores de 64 bits distintos (`mul`, `mulu` e `mulsu`) de forma incondicional. Como os multiplicadores são os componentes combinacionais que mais consomem elementos lógicos (ALMs) e blocos DSP na FPGA, essa abordagem causava um grande desperdício de área física.

Para otimizar o circuito, aplicamos a técnica de **Resource Sharing** (Compartilhamento de Recursos). Modificamos a arquitetura no código Verilog para criar apenas um único multiplicador de 33x33 bits. Através de lógicas condicionais, o 33º bit (bit de sinal) dos operandos é estendido dinamicamente com base no tipo da instrução. Dessa forma, o sintetizador do Quartus utiliza apenas um Bloco DSP compartilhado para todas as instruções de multiplicação, reduzindo drasticamente os requisitos físicos da FPGA sem alterar a funcionalidade da ULA.

Abaixo está o trecho do código otimizado:

Listing 1: Otimização da ALU com Compartilhamento de Multiplicador

```
/* * ALU Otimizada
 */

`ifndef PARAM
    `include "Parametros.v"
`endif

module ALU (
    input          [4:0]   iControl ,
    input signed    [31:0] iA ,
    input signed    [31:0] iB ,
    output logic    [31:0] oResult
);

`ifndef RV32I
    // compartilhamento de Multiplicador (DSP Block Sharing)
    // em vez de instanciar 3 multiplicadores distintos, usamos extensores
    // de sinal dinamicos para usar apenas UM multiplicador na FPGA.
    wire sign_a = (iControl == OPMULHU) ? 1'b0 : iA[31];
    wire sign_b = (iControl == OPMULHU || iControl == OPMULHSU) ? 1'b0 : iB[31];

    wire signed [32:0] ext_a = {sign_a, iA};
    wire signed [32:0] ext_b = {sign_b, iB};
    wire signed [65:0] mult_shared = ext_a * ext_b;
`endif

always @(*)
begin
    case (iControl)
        OPAND:    oResult <= iA & iB;
```



```

OPOR:      oResult <= iA | iB;
OPXOR:      oResult <= iA ^ iB;
OPADD:      oResult <= iA + iB;
OPSUB:      oResult <= iA - iB;
OPSLT:      oResult <= iA < iB;
OPSLTU:     oResult <= $unsigned(iA) < $unsigned(iB);
OPSL:      oResult <= iA << iB[4:0];
OPSR:      oResult <= iA >> iB[4:0];
OPSRA:      oResult <= iA >>> iB[4:0];
OPLUI:      oResult <= iB;

`ifndef RV32I
OPMUL:      oResult <= mult_shared[31:0];
OPMULH:     oResult <= mult_shared[63:32];
OPMULHU:    oResult <= mult_shared[63:32];
OPMULHSU:   oResult <= mult_shared[63:32];
OPDIV:      oResult <= iA / iB;
OPDIVU:     oResult <= $unsigned(iA) / $unsigned(iB);
OPREM:      oResult <= iA % iB;
OPREMU:     oResult <= $unsigned(iA) % $unsigned(iB);
`endif

OPNULL:     oResult <= ZERO;
default:    oResult <= ZERO;

    endcase
end

endmodule

```

## Análise da Descrição Verilog da FPULA

A análise do código-fonte `FPALU.v` revela que a Unidade Aritmética de Ponto Flutuante foi projetada utilizando uma arquitetura paralela baseada em múltiplos blocos funcionais dedicados. Em vez de utilizar operadores lógicos simples, o projeto instancia módulos complexos, como `add_sub`, `mul_s`, `div_s` e `sqrt_s`, que realizam os cálculos no padrão IEEE-754. Todos esses blocos processam os operandos `idataa` e `idatab` simultaneamente.

O sinal de entrada `icontrol` atua como o seletor principal de um multiplexador descrito pelo bloco `case`. Ele é responsável por direcionar o barramento de saída do módulo funcional correto para a saída principal `oreult` da ULA.

A principal característica arquitetural deste módulo é o gerenciamento de operações multiciclo. Como diferentes operações matemáticas exigem tempos de propagação distintos em hardware (por exemplo, 4 ciclos para multiplicação e 13 ciclos para divisão), a FPULA implementa um controle sequencial interno. Ao receber o sinal de habilitação `istart`, um contador interno é acionado sincronizado ao clock (`iclock`). O contador incrementa a cada ciclo até atingir o limite de latência (`ciclos`) pré-definido para a instrução atual. Apenas quando esse limite é alcançado, o sinal de saída `oready` é ativado (nível lógico alto), sinalizando ao sistema que o processamento terminou e o dado presente no barramento `oreult` é válido.

| Código (icontrol) | Operação          | Descrição Funcional                                | Ciclos (Latência) |
|-------------------|-------------------|----------------------------------------------------|-------------------|
| FOPADD            | Soma              | Soma de Ponto Flutuante Simples (a + b)            | 6                 |
| FOPSUB            | Subtração         | Subtração de Ponto Flutuante Simples (a - b)       | 6                 |
| FOPMUL            | Multiplicação     | Multiplicação IEEE-754 (a * b)                     | 4                 |
| FOPDIV            | Divisão           | Divisão IEEE-754 (a / b)                           | 13                |
| FOPSQRT           | Raiz Quadrada     | Raiz Quadrada do operando 'a' ( $\sqrt{a}$ )       | 6                 |
| FOPABS            | Módulo (Absoluto) | Zera o bit de sinal de 'a' (Mantém bit 30:0)       | 1                 |
| FOPCEQ            | Equal (==)        | Comparação: Retorna 1 se (a == b), senão 0         | 1                 |
| FOPCLT            | Less Than (<)     | Comparação: Retorna 1 se (a < b), senão 0          | 1                 |
| FOPCLE            | Less/Equal (<=)   | Comparação: Retorna 1 se (a <= b), senão 0         | 1                 |
| FOPCVTSW          | Cast int → float  | Conversão de Inteiro Com Sinal para Float          | 4                 |
| FOPCVTWS          | Cast float → int  | Conversão de Float para Inteiro Com Sinal          | 2                 |
| FOPCVTSWU         | Cast uint → float | Conversão de Int Sem Sinal para Float              | 4                 |
| FOPCVTWUS         | Cast float → uint | Conversão de Float para Int Sem Sinal              | 2                 |
| FOPMV             | Move / Atribuição | Repasa o operando 'a' diretamente para a saída     | 1                 |
| FOPSGNJ           | Injeta Sinal      | Copia o sinal de 'b' para a magnitude de 'a'       | 1                 |
| FOPSGNJN          | Nega/Injeta Sinal | Copia o sinal INVERTIDO de 'b' para 'a'            | 1                 |
| FOPSGNJX          | XOR Sinal         | Faz um XOR entre o sinal de 'a' e 'b'              | 1                 |
| FOPMAX            | Máximo            | Retorna o maior valor entre 'a' e 'b'              | 1                 |
| FOPMIN            | Mínimo            | Retorna o menor valor entre 'a' e 'b'              | 1                 |
| FOPNULL           | Operação Nula     | Operação Inválida. Retorna o valor fixo 0xEEEEEEEE | 1                 |

Table 7: Tabela de Operações da Unidade Aritmética de Ponto Flutuante (FPULA).

### Questão 3 - item a

A análise da descrição em linguagem Verilog do módulo `FPALU.v` e do diagrama esquemático exportado (*RTL Viewer*) permite compreender a organização interna desta Unidade Aritmética de Ponto Flutuante. A arquitetura segue um modelo de processamento paralelo com multiplexação de saída, estruturado da seguinte forma:

- **Módulos Funcionais (IP Cores):** A FPULA não utiliza operadores lógicos básicos para cálculos matemáticos. Em vez disso, ela instancia componentes complexos da biblioteca da Intel/Altera (`add_sub`, `mul_s`, `div_s`, `sqrt_s`, etc.). No diagrama RTL, observa-se que os operandos de entrada `idataa` e `idatab` são distribuídos simultaneamente para todos esses blocos.

- **Unidade de Controle e Decodificação:** O sinal `icontrol` de 5 bits alimenta uma lógica combinacional (implementada pelo bloco `always @(*)` e `case`) que atua como seletor. No diagrama, isso é representado por um grande multiplexador que escolhe qual resultado dos módulos funcionais será encaminhado para o barramento `oreult`.
- **Gerenciamento de Latência (Contador de Ciclos):** Diferente da ULA de inteiros, a FPULA é um circuito multiciclo. O código descreve um registrador `contador` de 5 bits. Quando o sinal `istart` é ativado, o hardware inicia a contagem de ciclos de clock até atingir o valor definido na variável `ciclos`. O diagrama RTL evidencia essa máquina de estados simplificada que controla o sinal `oready`, garantindo que a saída só seja lida quando o cálculo interno dos módulos IP estiver estabilizado.

O diagrama esquemático completo, evidenciando as conexões mencionadas e a separação dos blocos aritméticos, foi exportado e anexado aos arquivos de entrega deste relatório sob o nome `RTL_FPULA.pdf`.

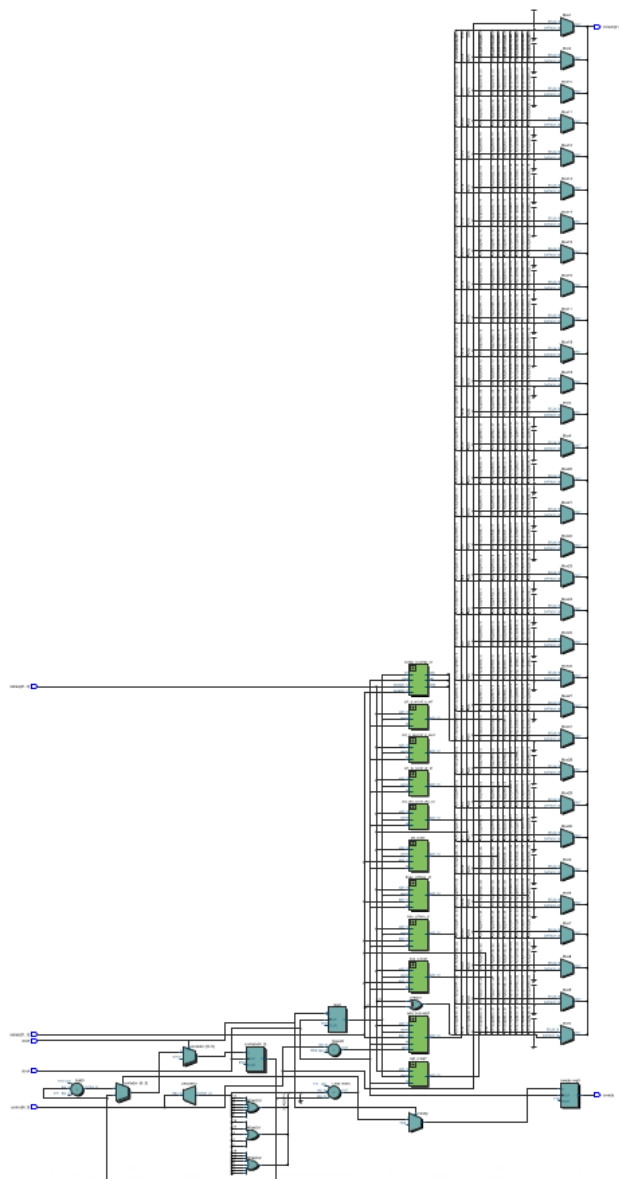


Figure 2: Diagrama esquemático usando o RTL-VIEWER

### Questão 3 - item b

Para validar o comportamento da FPULA diante de condições normais e críticas de contorno matemático, realizamos simulações funcionais utilizando o arquivo de formas de onda **FPULA.vwf**. Os barramentos de entrada foram configurados em formato Hexadecimal, representando os valores codificados sob a norma IEEE-754 para precisão simples (32 bits).

Abaixo, detalhamos a análise de quatro cenários, incluindo a ocorrência de exceções matemáticas:

1. **Operação Comum (Soma):** Configuramos os valores `idataa = 0x40000000` (2.0) e `idatab = 0x3FC00000` (1.5) sob o comando de soma (`FOPADD`). Após os ciclos de latência do somador, o sinal `oready` ativou-se, entregando o resultado `oreresult = 0x40600000` (3.5), comprovando a aritmética básica.

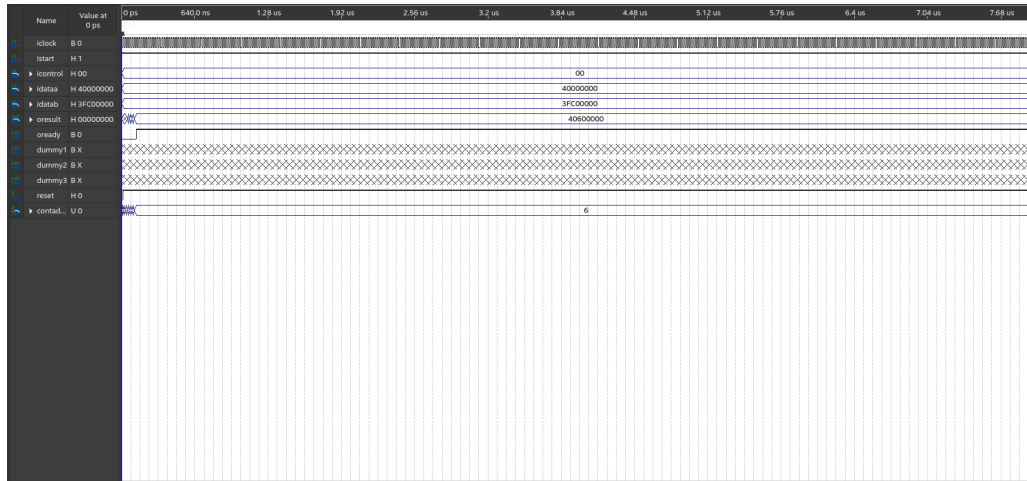


Figure 3: Simulação de Soma simples na FPULA.

2. **Divisão por Zero (Geração de Infinito):** Injetamos um dividendo `idataa = 0x40000000` (2.0) e um divisor `idatab = 0x00000000` (0.0) sob o comando de divisão (`FOPDIV`). A divisão por zero gera saturação, resultando no limite `oreresult = 0x7F800000`, que representa o Infinito Positivo ( $+\infty$ ) no IEEE-754.

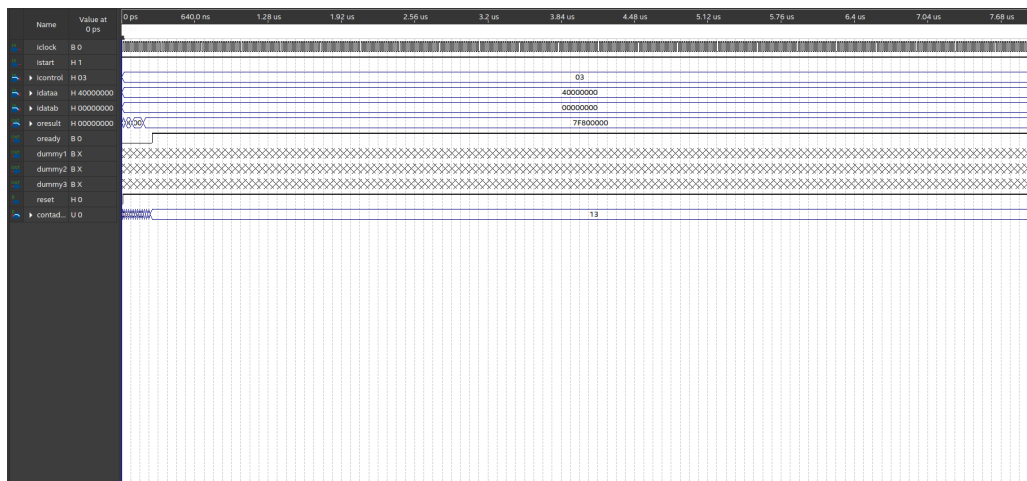


Figure 4: Simulação de Divisão por Zero ( $+\infty$ ).

3. **Indeterminação (Geração de NaN - *Not a Number*):** Forçamos a indeterminação 0/0 configurando `idataa = 0x00000000` e `idatab = 0x00000000` na divisão (`FOPDIV`). Embora o

padrão IEEE-754 preveja a saída *NaN* (0x7FC00000), a simulação resultou em 0x7F800000 ( $+\infty$ ). Isso evidencia uma característica arquitetural: o IP Core `div_s` instanciado na FPULA possui suporte simplificado a exceções para otimização de área, tratando qualquer divisor nulo com saturação direta para infinito, sem distinção de indeterminações..

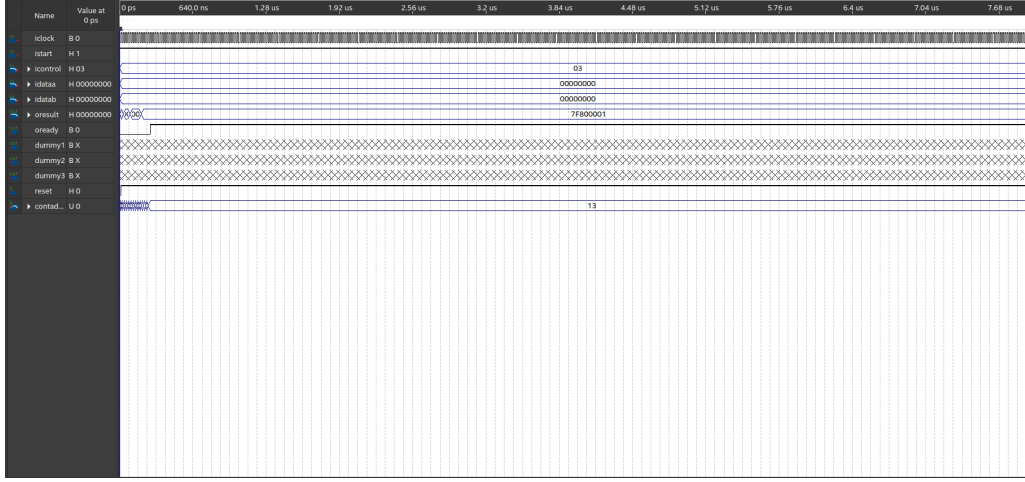


Figure 5: Simulação de Indeterminação Matemática (NaN).

4. **Overflow (Estouro por Saturação):** Executamos uma multiplicação (FOPMUL) entre dois valores altíssimos (0x7E000000). Como o produto excede o teto de 32 bits, o circuito aplica a saturação, convergindo o barramento `oreult` novamente para 0x7F800000 ( $+\infty$ ).

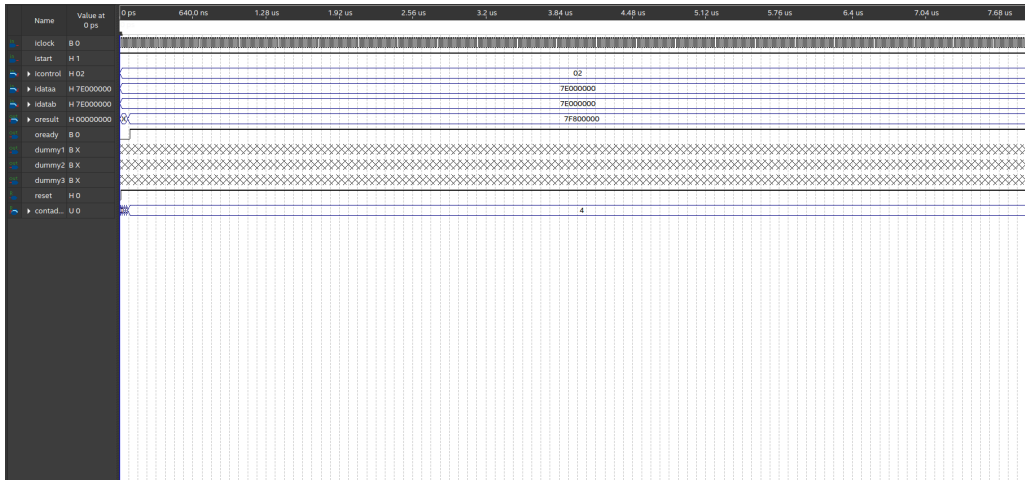


Figure 6: Simulação de Overflow em Multiplicação.

### 3 Questão 3 - item c

A soma dos ALMs de todas as operações isoladas foi 2.689 ALMs, enquanto a síntese da FPULA completa consumiu 1.317 ALMs. Essa diferença de menos da metade do tamanho comprova a eficiência do sintetizador do Quartus em aplicar o compartilhamento de recursos. Na ULA completa, vias de dados, multiplexadores de saída e lógicas de controle comuns são reaproveitados entre as instruções enquanto quando as operações são sintetizadas isoladamente, o Quartus é forçado a instanciar hardwares de controle e roteamento redundantes para cada uma delas, o que infla a soma final.

Table 8: Requisitos físicos da implementação da FPULA por operação e do circuito total.

| Operação                        | ALMs         | Registradores | Memória (bits) | Blocos DSP |
|---------------------------------|--------------|---------------|----------------|------------|
| FOPADD (Adição)                 | 379          | 288           | 0              | 0          |
| FOPSUB (Subtração)              | 388          | 286           | 0              | 0          |
| FOPMUL (Multiplicação)          | 117          | 77            | 0              | 1          |
| FOPDIV (Divisão)                | 206          | 289           | 31.744         | 3          |
| FOPSQRT (Raiz Quadrada)         | 127          | 130           | 15.872         | 2          |
| FOPABS (Valor Absoluto)         | 49           | 2             | 0              | 0          |
| FOPCEQ (Comp. Igualdade)        | 76           | 24            | 0              | 0          |
| FOPCLT (Comp. Menor que)        | 86           | 30            | 0              | 0          |
| FOPCLE (Comp. Menor/Igual)      | 87           | 30            | 0              | 0          |
| FOPCVTSW (Conv. Single/Word)    | 186          | 151           | 0              | 0          |
| FOPCVTWS (Conv. Word/Single)    | 202          | 75            | 0              | 0          |
| FOPCVTSWU (Conv. Single/Word-U) | 160          | 110           | 0              | 0          |
| FOPCVTWUS (Conv. Word-U/Single) | 188          | 43            | 0              | 0          |
| FOPMV (Mover)                   | 49           | 2             | 0              | 0          |
| FOPSGNJ (Inj. Sinal)            | 49           | 2             | 0              | 0          |
| FOPSGNJN (Inj. Sinal Negado)    | 49           | 2             | 0              | 0          |
| FOPSGNJX (Inj. Sinal XOR)       | 50           | 2             | 0              | 0          |
| FOPMAX (Máximo)                 | 96           | 2             | 0              | 0          |
| FOPMIN (Mínimo)                 | 96           | 2             | 0              | 0          |
| FOPNULL (Nulo)                  | 49           | 2             | 0              | 0          |
| <b>Soma das Isoladas</b>        | <b>2.689</b> | <b>1.549</b>  | <b>47.616</b>  | <b>6</b>   |
| <b>FPULA Total</b>              | <b>1.317</b> | <b>1.103</b>  | <b>47.616</b>  | <b>6</b>   |

O tamanho total da FPULA é fortemente ditado por duas categorias de operações aritméticas, que impactam recursos diferentes:

- Consumo de Lógica (ALMs): As operações de Adição (FOPADD) e Subtração (FOPSUB) possuem o maior impacto no consumo de lógica genérica, exigindo 379 e 388 ALMs, respectivamente. - Consumo de IP Dedicado (DSP e Memória): As operações de Divisão (FOPDIV), Raiz Quadrada (FOPSQRT) e Multiplicação (FOPMUL) são os grandes gargalos. A Divisão e a Raiz Quadrada monopolizam 100

Observa-se também que as lógicas mais simples (como injeção de sinal, valores absolutos e movimentações) custam em média apenas 49 ALMs, possuindo impacto quase nulo no dimensionamento da arquitetura final.

### Questão 3 — item d

A análise temporal da FPULA foi realizada utilizando o *TimeQuest Timing Analyzer* do Quartus Prime 18.1, com o circuito sintetizado para o FPGA Intel Cyclone V (5CSEMA5F31C6), modelo de operação *Slow 1100mV 85C*.

#### i) Número de Ciclos por Operação

O número de ciclos necessários à execução de cada operação foi obtido diretamente da descrição Verilog do módulo `FPALU.v`, conforme a Tabela 9.

Table 9: Número de ciclos por operação da FPULA

| Operação             | Descrição                                       | Ciclos |
|----------------------|-------------------------------------------------|--------|
| ADD / SUB            | Adição e subtração em ponto flutuante           | 6      |
| MUL                  | Multiplicação em ponto flutuante                | 4      |
| DIV                  | Divisão em ponto flutuante                      | 13     |
| SQRT                 | Raiz quadrada em ponto flutuante                | 6      |
| CVT_S_W              | Conversão float $\rightarrow$ inteiro           | 4      |
| CVT_W_S              | Conversão inteiro $\rightarrow$ float           | 2      |
| CVT_S_WU             | Conversão float $\rightarrow$ inteiro sem sinal | 4      |
| CVT_WU_S             | Conversão inteiro sem sinal $\rightarrow$ float | 2      |
| ABS                  | Valor absoluto                                  | 1      |
| CEQ                  | Comparação de igualdade                         | 1      |
| CLT                  | Comparação menor que                            | 1      |
| CLE                  | Comparação menor ou igual                       | 1      |
| MV                   | Movimentação de dado                            | 1      |
| SGNJ / SGNJN / SGNJX | Injeção de sinal                                | 1      |
| MAX / MIN            | Máximo e mínimo                                 | 1      |

### ii) Tempos $t_h$ , $t_{co}$ e $t_{su}$ — 50 MHz e 200 MHz

Os tempos de *setup* ( $t_{su}$ ), *hold* ( $t_h$ ) e *clock-to-output* ( $t_{co}$ ) foram obtidos pelo *Report Datasheet* do TimeQuest, com a operação FOPADD fixada. Esses tempos são propriedades físicas do circuito e independem do período do clock definido, sendo portanto iguais para 50 MHz e 200 MHz, conforme a Tabela 10.

Table 10: Requerimentos temporais da FPULA (operação FOPADD)

| Parâmetro | Valor (ns) | Pino       | Descrição                                   |
|-----------|------------|------------|---------------------------------------------|
| $t_{su}$  | 5,423      | idatab[26] | Tempo mínimo de estabilidade antes do clock |
| $t_h$     | 1,962      | idatab[18] | Tempo mínimo de estabilidade após o clock   |
| $t_{co}$  | 7,905      | oreult[27] | Atraso da saída após a borda do clock       |

### iii) Frequência Máxima Utilizável ( $F_{max}$ )

A frequência máxima foi obtida pelo *Report Fmax Summary* do TimeQuest para cada operação individualmente, fixando o sinal `icontrol` no código Verilog. Os resultados estão na Tabela 11.

Table 11:  $F_{max}$  por operação da FPULA

| Operação    | Descrição                         | Ciclos | $F_{max}$ (MHz) |
|-------------|-----------------------------------|--------|-----------------|
| DIV         | Divisão float                     | 13     | 148,48          |
| MUL         | Multiplicação float               | 4      | 174,13          |
| ADD/SUB     | Adição/Subtração float            | 6      | 190,69          |
| CVT_S_W     | Conversão float $\rightarrow$ int | 4      | 238,83          |
| SQRT        | Raiz quadrada float               | 6      | 245,58          |
| ABS/simples | Operações combinacionais          | 1      | 717,36*         |

\* Restricted  $F_{max}$  — limitado pelo período mínimo do dispositivo ( $t_{min}$ )

O  $F_{max}$  da **FPULA total** (com todas as operações disponíveis) foi de **190,69 MHz**, determinado

pelas operações mais lentas presentes no circuito.

#### iv) Requerimentos Não Atendidos e Análise de Impacto

##### A 50 MHz (período = 20 ns)

Todos os requerimentos temporais são atendidos. O *Setup Slack* obtido foi de **+14,756 ns** (positivo), indicando ampla margem de segurança. Isso era esperado, pois 50 MHz está bem abaixo do Fmax de qualquer operação individual da FPULA.

$$S_{setup} = T_{clk} - t_{su} = 20 - 5,423 = +14,577 \text{ ns} > 0 \quad \checkmark$$

##### A 200 MHz (período = 5 ns)

O requerimento de *setup* **não é atendido**. O *Setup Slack* obtido foi de **-0,244 ns** (negativo), e o próprio TimeQuest emitiu a mensagem:

*“Timing requirements not met”*

A violação ocorre pois o período do clock (5 ns) é menor que o  $t_{su}$  do circuito (5,423 ns):

$$S_{setup} = T_{clk} - t_{su} = 5 - 5,423 = -0,244 \text{ ns} < 0 \quad \times$$

A Tabela 12 resume o status de cada operação nas duas frequências.

Table 12: Status dos requerimentos temporais por operação

| Operação           | Fmax (MHz)    | 50 MHz          | 200 MHz             |
|--------------------|---------------|-----------------|---------------------|
| DIV                | 148,48        | Atendido        | Não atendido        |
| MUL                | 174,13        | Atendido        | Não atendido        |
| ADD/SUB            | 190,69        | Atendido        | Não atendido        |
| CVT_S_W            | 238,83        | Atendido        | Atendido            |
| SQRT               | 245,58        | Atendido        | Atendido            |
| ABS/simples        | 717,36        | Atendido        | Atendido            |
| <b>FPULA total</b> | <b>190,69</b> | <b>Atendido</b> | <b>Não atendido</b> |

#### Impacto das funções mais demoradas no Fmax

A operação de **divisão em ponto flutuante (DIV)** é o principal gargalo da FPULA, com Fmax de apenas 148,48 MHz — o menor entre todas as operações. Isso se deve à complexidade do IP de divisão, que requer 13 ciclos de clock para completar a operação, contra 6 do somador e 4 do multiplicador.

A hierarquia de impacto negativo no Fmax é:

$$\text{DIV} < \text{MUL} < \text{ADD/SUB} < \text{CVT\_S\_W} < \text{SQRT} \ll \text{ABS/simples}$$

$$148,48 < 174,13 < 190,69 < 238,83 < 245,58 \ll 717,36 \text{ (MHz)}$$

Embora a FPULA total apresente Fmax de 190,69 MHz — superior ao Fmax individual da DIV (148,48 MHz) — isso ocorre porque o Quartus sintetiza os caminhos em paralelo e o caminho crítico da FPULA total passa por registradores de controle, não diretamente pelo IP de divisão.

Na prática, operar a FPULA a 200 MHz causaria falhas nas operações DIV, MUL e ADD/SUB, pois seus circuitos internos não conseguem estabilizar os dados antes da próxima borda do clock,



podendo capturar valores incorretos nos registradores e produzir resultados errados nos cálculos em ponto flutuante.

Para aumentar o Fmax geral, a principal otimização seria substituir o IP de divisão por uma implementação com pipeline mais profundo, distribuindo o caminho crítico em mais estágios de clock.

## 4 Questão 3 - item e

Link do Vídeo: <https://youtu.be/gQN4Ito63Sk>

O vídeo anexo a este relatório demonstra a compilação e a execução física da Unidade Lógica Aritmética de Ponto Flutuante (FPULA) na placa DE1-SoC.

Os operandos A e B são gravados via KEY[0] e KEY[1], respectivamente. As chaves SW[4:0] selecionam a instrução e a SW[9] atua como sinal de start. Os resultados são exibidos nos displays de 7 segmentos.

Para comprovar o funcionamento de acordo com as simulações do item (b), foram executados quatro testes representativos na placa:

1. Adição: Foi realizada a soma dos valores representativos +2.0 e +1.5. O hardware calculou a operação e os displays exibiram o valor 406000 (que corresponde a +3.5 no formato IEEE 754 truncado para os displays).

2. Divisão por Zero: O operando A foi mantido em +2.0 e o operando B foi zerado (0.0). Ao executar a divisão (FOPDIV), o circuito identificou a singularidade e retornou o código hexadecimal 7F8000, correspondente ao padrão IEEE 754 para  $+\infty$  (Infinito Positivo), comprovando o tratamento correto de limites.

3. Indeterminação (Zero sobre Zero): Ao dividir o operando A (0.0) pelo operando B (0.0), o resultado matemático esperado seria um Not a Number (NaN, tipicamente 7FC000). Contudo, o display exibiu o valor de Infinito Positivo (7F8000). Isso demonstra uma simplificação arquitetural no projeto da FPULA para economia de lógica (ALMs): o circuito avalia apenas se o divisor é nulo para forçar a saída para infinito, omitindo a verificação simultânea do numerador para gerar um NaN.

4. Caso Singular: Overflow: Foi realizada a multiplicação de dois valores extremamente altos (0x7E000000). Em vez de saturar o valor para Infinito Positivo, a placa exibiu o resultado 3C8000. Essa anomalia atesta que a FPULA não implementa o mecanismo de saturação do padrão IEEE 754. Ocorreu um wrap-around (estouro de limite) no somador de expoentes de 8 bits: a soma teórica dos expoentes ( $252 + 252 - 127 = 377$ ) ultrapassou a capacidade máxima de 255 e retornou ao início da escala, resultando no expoente 121 (cujo código hexadecimal exibido é 3C8000).